

Compte Rendu : TD2 – Bank Churners

Preprocessing et Préparation des Données

Ayman Chabbaki

Groupe : G1

June 6, 2026

Introduction

Ce compte rendu présente les différentes étapes du TD2 portant sur le diagnostic, le nettoyage et la préparation du jeu de données *BankChurners*. L'objectif de ce projet est de préparer les données pour la modélisation de l'attrition client (churn) en gérant les valeurs manquantes simulées, les outliers, l'encodage des variables catégorielles et la normalisation des variables numériques.

1 Diagnostic et Simulation des Manquants

1.1 Profil Initial du Dataset

Le jeu de données original comprend 10 127 clients et 19 variables (après suppression des deux dernières colonnes inutiles). On dénombre 10 variables de type `int64`, 6 de type `object` (variables catégorielles) et 3 de type `float64`. La variable cible `Attrition_Flag` indique un taux de churn d'environ 16,1%, ce qui révèle un déséquilibre des classes qu'il faudra prendre en compte lors de l'évaluation du modèle.

1.2 Mécanismes de Données Manquantes (MCAR)

Des valeurs manquantes ont été introduites aléatoirement (mécanisme MCAR - Missing Completely At Random) sur trois colonnes :

- `Gender` : 8% (810 valeurs)
- `Education_Level` : 12% (1215 valeurs)
- `Total_Trans_Amt` : 5% (506 valeurs)

Le mécanisme MCAR est le plus simple à traiter car l'absence de donnée ne dépend d'aucune autre variable. L'utilisation du paramètre `replace=False` lors de la simulation garantit qu'un même index ne soit pas sélectionné plusieurs fois.

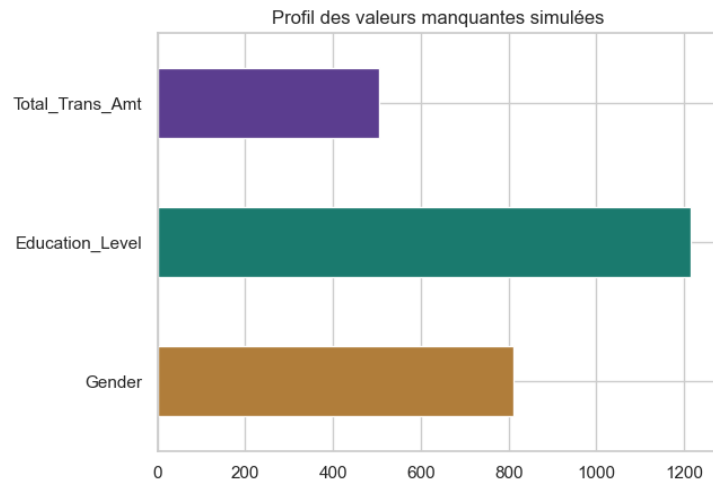


Figure 1: Profil des valeurs manquantes simulées

1.3 Détection des Outliers (Méthode IQR)

L'analyse par la méthode de l'écart interquartile (IQR) montre la présence d'outliers significatifs dans les variables financières. Par exemple, `Credit_Limit` présente environ 9,7% d'outliers et `Total_Trans_Amt` 8,8%.

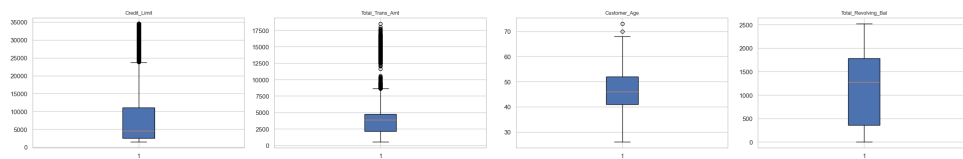


Figure 2: Boxplots des variables numériques illustrant les outliers

Pour ces variables financières avec une forte asymétrie et de nombreux outliers, un `MinMaxScaler` n'est pas adapté car il écraserait la majorité des valeurs dans une petite plage. Le `RobustScaler` est le choix privilégié ici.

2 Imputation des Valeurs Manquantes

2.1 Imputation Numérique

L'imputation sur la variable `Total_Trans_Amt` a été testée avec trois stratégies : moyenne, médiane et constante (0).

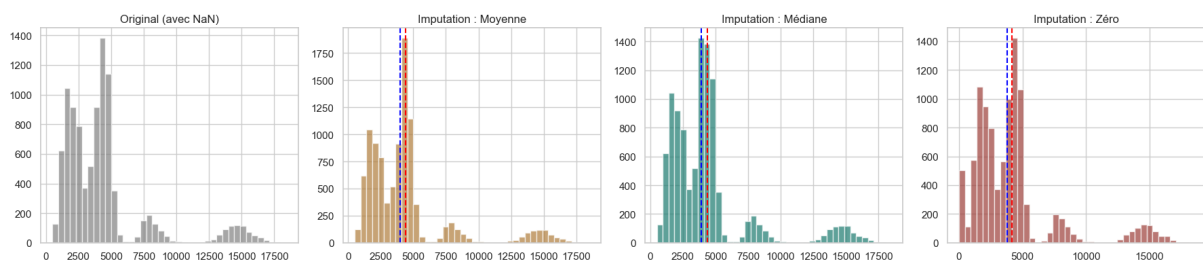


Figure 3: Comparaison des stratégies d'imputation numérique

Choix effectué : La **médiane** est recommandée car elle est robuste aux valeurs extrêmes

et préserve mieux la distribution d'une variable asymétrique comme le montant des transactions. L'imputation par zéro crée un pic artificiel injustifié métier, et la moyenne réduit artificiellement la variance en étant trop influencée par les outliers.

2.2 Imputation Catégorielle

Pour **Gender** (binaire, 53/47), l'imputation par le **mode** (stratégie la plus fréquente) est appropriée. Cependant, pour **Education_Level**, qui contenait déjà des valeurs 'Unknown' (17,5%) avant même la simulation, il est plus cohérent de remplacer les NaN par la **constante 'Unknown'** pour ne pas fausser la répartition naturelle et regrouper toute l'information manquante dans une catégorie existante.

3 Encodage des Variables Catégorielles

L'encodage des variables a été réalisé en respectant leur nature sémantique :

- **Cible et Binaires** : Mappage manuel (**Attrited Customer** = 1, **Existing Customer** = 0). L'Attrition est la classe positive (1) car c'est l'événement que la banque cherche à détecter.
- **Ordinal Encoding** : **Education_Level** et **Income_Category** ont été encodées selon un ordre logique croissant (ex: Uneducated → Doctorate). Le taux de churn a été vérifié pour s'assurer qu'il suit globalement une tendance avec cet ordre.
- **One-Hot Encoding** : La variable nominale **Card_Category** a été transformée en *dummies*. Le paramètre `drop_first=True` a été utilisé pour éviter le "dummy trap" (multicolinéarité parfaite).

4 Normalisation et Pipeline Final

4.1 Choix des Scalers

Une comparaison sur **Credit_Limit** montre que le **RobustScaler** est le plus efficace pour gérer les outliers sans écraser la distribution principale, contrairement au **StandardScaler** ou au **MinMaxScaler**.

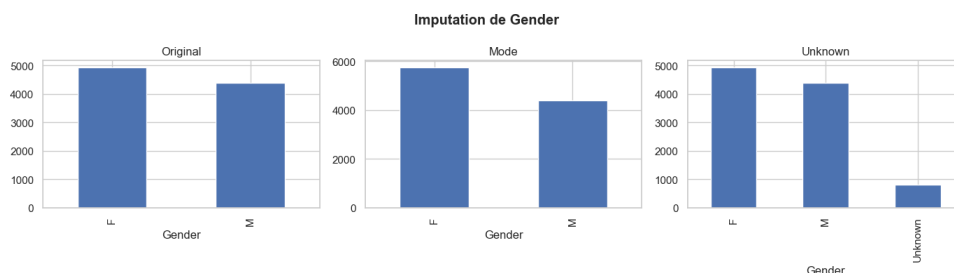


Figure 4: Impact des différents scalers sur la distribution avec outliers

4.2 ColumnTransformer et Prévention du Leakage

Pour garantir une méthodologie rigoureuse, les données ont été séparées (`train_test_split`) **avant** l'assemblage du pipeline. Le **ColumnTransformer** a été ajusté (`fit`) uniquement sur le jeu d'entraînement, puis appliqué (`transform`) au test, afin d'éviter toute fuite de données (data leakage). Le pipeline a été exporté sous `churn_preprocessor.pkl`.

5 Synthèse et Tableau Récapitulatif

En conclusion, le nettoyage et la préparation du dataset BankChurners ont impliqué un diagnostic minutieux des distributions. Les transformations ont été choisies de manière argumentée et compilées dans un Pipeline robuste aux anomalies et aux nouvelles données.

Variable(s)	Stratégie d'Imputation	Stratégie d'Encodage/Scaling
Total_Trans_Amt, Credit_Limit	Médiane (robuste aux outliers)	RobustScaler
Customer_Age, Months_on_book	Médiane	RobustScaler
Gender, Marital_Status	Mode (<i>most_frequent</i>)	OrdinalEncoder (Mappage Binaire)
Education_Level, Income	Constante ('Unknown')	OrdinalEncoder (Ordre logique)
Card_Category	Mode (<i>most_frequent</i>)	OneHotEncoder (drop='first')

Table 1: Récapitulatif des choix de preprocessing